

Video Game Film Adaptations: Factors in Box Office and Critical Reception

Harsh, Charlie, Mason

2026-06-24

Introduction & Motivation

Video game film adaptations sit in a strange spot. The genre has been around since the 1980s and most of it has been bad. Really bad. But every once in a while, something breaks through. In 2023, *The Super Mario Bros. Movie* pulled in \$1.3 billion. That same dataset includes 1993's *Super Mario Bros.*, widely considered one of the worst films ever made. The same IP. Three decades apart. Two completely different outcomes.

The track record is rough. *Street Fighter* (1994) tanked. *Monster Hunter* (2020) came and went. The *Resident Evil* reboot from 2021 didn't land either. But some titles did fine. Some did great. What explains the difference?

We wanted to look at three things. Do critics and audiences agree on these films? Does a big publisher with deep pockets automatically mean a big box office return? And do these franchises burn out over time the way standard Hollywood series do? The dataset goes back to 1986, which gives us enough runway to look at all three questions.

Research Questions

We focused on three questions:

1. **Critic-Audience Agreement Over Time:** Do critics and audiences agree on video game films? Has the gap between them changed?
2. **Publisher Size and Box Office Performance:** Does the size of the game publisher predict how much money its film adaptation makes?
3. **Franchise Fatigue:** Do video game film franchises make less money and get worse reviews with each new installment?

Data Provenance

Primary Dataset

Our main data source is the TidyTuesday *Game Films* dataset from June 9, 2026, hosted on the `rfordatascience/tidytuesday` GitHub repo. The data was scraped from Wikipedia’s “List of films based on video games.” Each row is one film adaptation.

The raw dataset has 439 rows and 19 variables: title, release date, category (theatrical, direct-to-video, TV, etc.), original game publisher, worldwide box office with currency label, budget bounds, Rotten Tomatoes score, Metacritic score, CinemaScore audience grade, and distributor info.

Supplementary Data

We added two external sources:

- **CPI-U Inflation Data:** U.S. Bureau of Labor Statistics CPI-U annual averages, 1986 to 2024. We used these to adjust all dollar amounts to 2024 dollars so we could compare box office across decades.
- **Publisher Revenue Data:** Wikipedia’s “List of largest video game companies by revenue,” scraped with the `rvest` package. This gives estimated yearly revenue for major publishers, which we used as a proxy for company size.

Data Wrangling & Cleaning

We ran several cleaning steps to get the data ready. We describe each one below. The matching code is in the Code Appendix.

Variable Selection

We kept all the original variables. Each one was used in at least one of our questions. The main ones: `title` for finding franchises, `release_date` for time trends, `original_game_publisher` for the publisher-size question, the box office and budget columns for financial analysis, and `rotten_tomatoes`, `metacritic`, and `cinema_score` for the critic-audience comparison.

Currency Standardization

The raw monetary values use their original currencies, marked by a separate currency column. We converted everything to USD with the exchange rates below:

Table 1: Exchange rates used for currency standardization

Currency	USD per Unit
\$	1.0000
¥	0.0091
£	1.3000
HK\$	0.1280
NT\$	0.0330

We joined this table to the main dataset on both `worldwide_box_office_currency` and `budget_currency`. This created new USD columns: `worldwide_box_office_usd`, `budget_low_usd`, and `budget_high_usd`.

Inflation Adjustment

Comparing box office across decades means adjusting for inflation. We added a CPI-U table, matched on the year from `release_date`, and multiplied all USD amounts by `CPI_2024 / CPI_year`. This puts everything in constant 2024 dollars. A dollar earned in 1993 counts the same as a dollar earned in 2024.

Film Category Filtering

The dataset covers several distribution types, including direct-to-video and TV specials. These tend to have spotty financial and review data. We filtered to only theatrical releases since those have the most complete records.

CinemaScore Conversion

The `cinema_score` column stores letter grades (A+ to F) as text, plus "N/A" for films with no score. To run numbers on it, we mapped each letter to a 0 to 100 scale. Films with "N/A" got NA and were left out of any analysis that needs audience scores.

Final Dataset Overview

After all cleaning, the dataset is theatrical-release video game films with standardized USD financials adjusted to 2024 dollars and a numeric cinema score. Here is a summary:

Table 2: Overview of the cleaned dataset after all preparation steps

Total Films	Films with Box Office Data	Films with RT Score	Films with Metacritic	Films with CinemaScore
246	89	73	63	51

Exploratory Data Analysis

We did a lot of EDA to dig into the three questions. This section walks through the main tables and figures. Each one is numbered, captioned, and discussed below.

Descriptive Statistics

Table 3 shows the mean, median, standard deviation, min, and max for the key numeric variables.

Table 3: Summary Statistics for Key Numeric Variables

Variable	Mean	Median	SD	Min	Max
Box Office (adj. USD)	152527625.6	63643362.7	208588976.1	63327.9	1400977007.7
Budget High (adj. USD)	81431922.1	70412468.1	62437356.0	2385551.3	287666208.2
Rotten Tomatoes	35.2	30.0	24.5	0.0	100.0
Metacritic	37.4	36.0	14.0	9.0	88.0
CinemaScore	63.2	66.7	22.8	0.0	91.7

Box office values span from near zero to over a billion dollars after inflation adjustment. The distribution is heavily right-skewed, which is common for entertainment industry money. Critic scores (Rotten Tomatoes and Metacritic) average in the 40 to 50 range, so video game films mostly get lukewarm to bad reviews. CinemaScore audience grades land around 60 on our 0 to 100 scale. That is a lot higher than critic scores, which points toward the gap we look at in Q1.

Q1: Critic-Audience Agreement Over Time

Our first question: do critics and audiences agree on video game films? Has the gap changed since 1986? We compared three score types: Rotten Tomatoes (critic aggregate), Metacritic (critic aggregate), and CinemaScore (audience survey, converted to numeric).

Figure 1 plots all three against release date with loess-smoothed trend lines and 95% confidence bands.

```
`geom_smooth()` using formula = 'y ~ x'
```

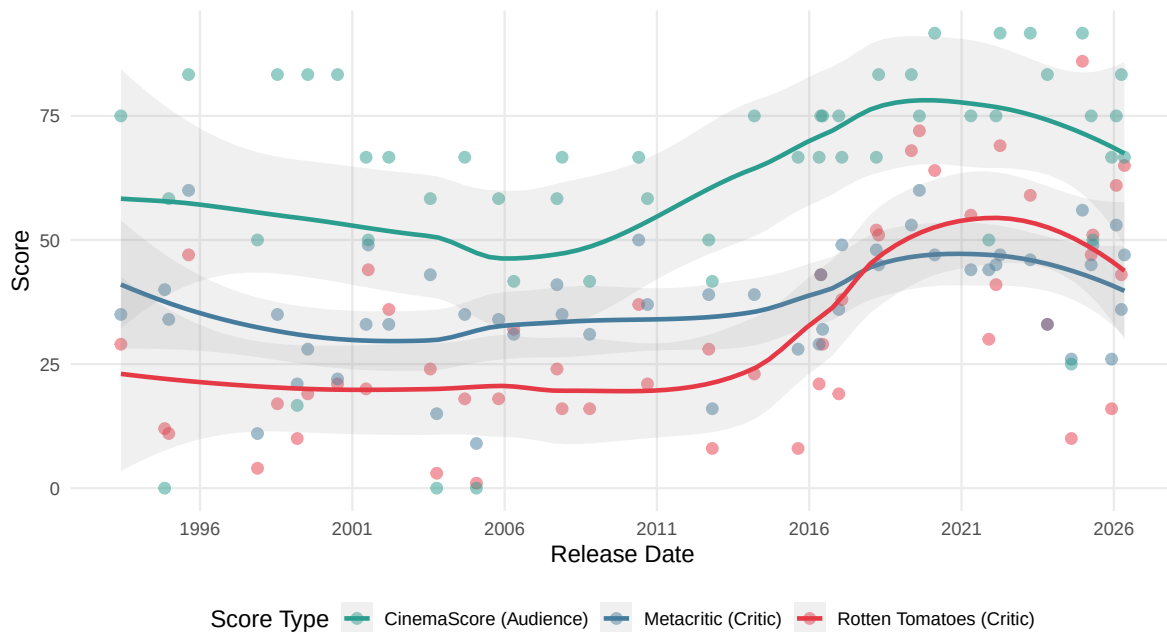


Figure 1: Critic and audience scores over time for video game film adaptations, 1986 to 2024. Loess-smoothed trends with 95% confidence bands.

A few things stand out in Figure 1. CinemaScore (audience) ratings sit above both critic metrics across the whole timeline. The loess CinemaScore trend runs between 60 and 70. Rotten Tomatoes and Metacritic cluster in the 40 to 55 range. Audiences rate these films 10 to 20 points higher than critics, and this has been true since the start. All three score types also show a slow upward trend after about 2010. That might be because studios started taking video game adaptations more seriously after the early failures, though we have no direct way to test that.

To measure the gap directly, Figure 2 shows the difference between average critic scores (mean of Rotten Tomatoes and Metacritic) and CinemaScore. Points above zero mean critics scored higher. Points below mean audiences scored higher.

```
`geom_smooth()` using formula = 'y ~ x'
```

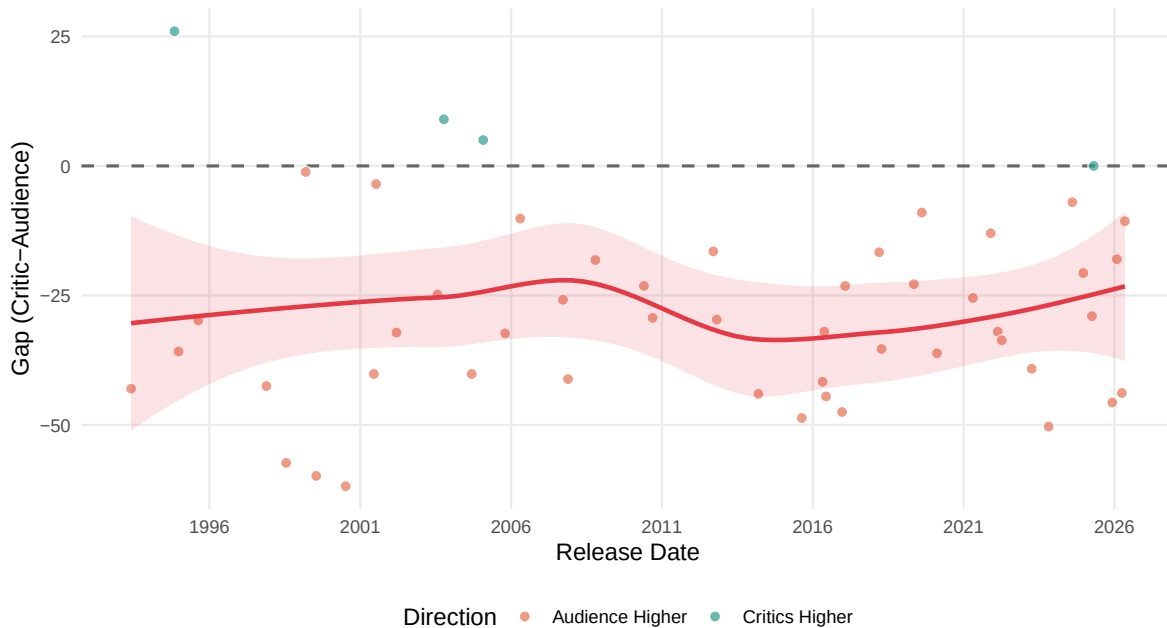


Figure 2: Critic-audience score gap over time. Positive values indicate critics rated films higher than audiences; negative values indicate the reverse.

Figure 2 shows most films fall below zero. Audiences rate these movies higher than critics almost every time. The loess trend suggests the gap has narrowed a bit since 2010, though the confidence band gets wider in recent years because there are fewer data points. Four decades of negative gaps point to a real split between how paid critics and regular moviegoers judge video game films.

Key findings for Q1: Audiences rate video game films higher than critics by about 10 to 20 points. That gap has narrowed a little since 2010. All score types trended up over the same period, so the genre might be getting better, or at least better received.

Q2: Publisher Size and Box Office Performance

Our second question: do bigger publishers produce film adaptations that make more money? The idea makes sense. A company like Nintendo has far more money than a smaller publisher, so you might expect their films to get bigger budgets, better marketing, and higher returns.

To check, we scraped Wikipedia’s list of the largest video game companies by yearly revenue and joined it with our film dataset. After matching and dropping rows with missing data, we had 36 films across 8 publishers.

Table 4 shows the aggregated numbers for publishers with at least three films in the matched set, ranked by mean inflation-adjusted box office.

Table 4: Publisher Performance: Aggregated Box Office and Critic Metrics (for 7 matched films)

company	# Films	Median Box Office (M USD)	Mean Box Office
Sega	4	411.0	
Eidos	3	266.8	
Ubisoft	4	157.8	
Capcom	15	51.1	
Square Enix	4	58.8	
Nintendo | The Pokémon Company	24	52.3	
Type-Moon	3	24.3	

Figure 3 is our 3+ variable figure. It packs four variables into one plot: publisher revenue (x axis, log scale), inflation-adjusted box office (y axis, log scale), publisher identity (color), and film budget (point size).

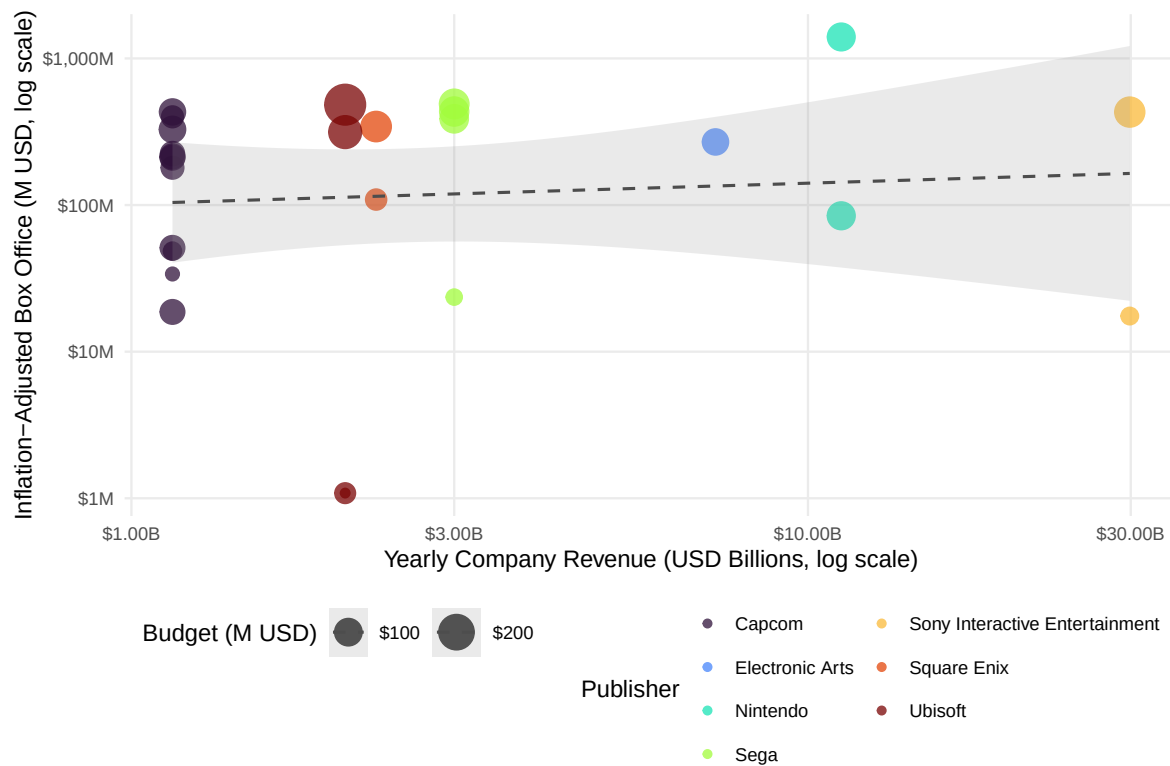
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

```
`geom_smooth()` using formula = 'y ~ x'
```

Warning: The following aesthetics were dropped during statistical transformation: size.
i This can happen when ggplot fails to infer the correct grouping structure in the data.
i Did you forget to specify a `group` aesthetic or to convert a numerical variable into a factor?

The dashed LM line in Figure 3 shows a very weak link between publisher revenue and box office. The two biggest earners in the sample are both Nintendo’s *Super Mario* films, and Nintendo is one of the largest publishers by revenue. But most publishers cluster in a narrow band of box office numbers no matter their size. Figure 4 shows the same pattern another way with a ridgeline density plot of box office distributions.

Picking joint bandwidth of 0.543



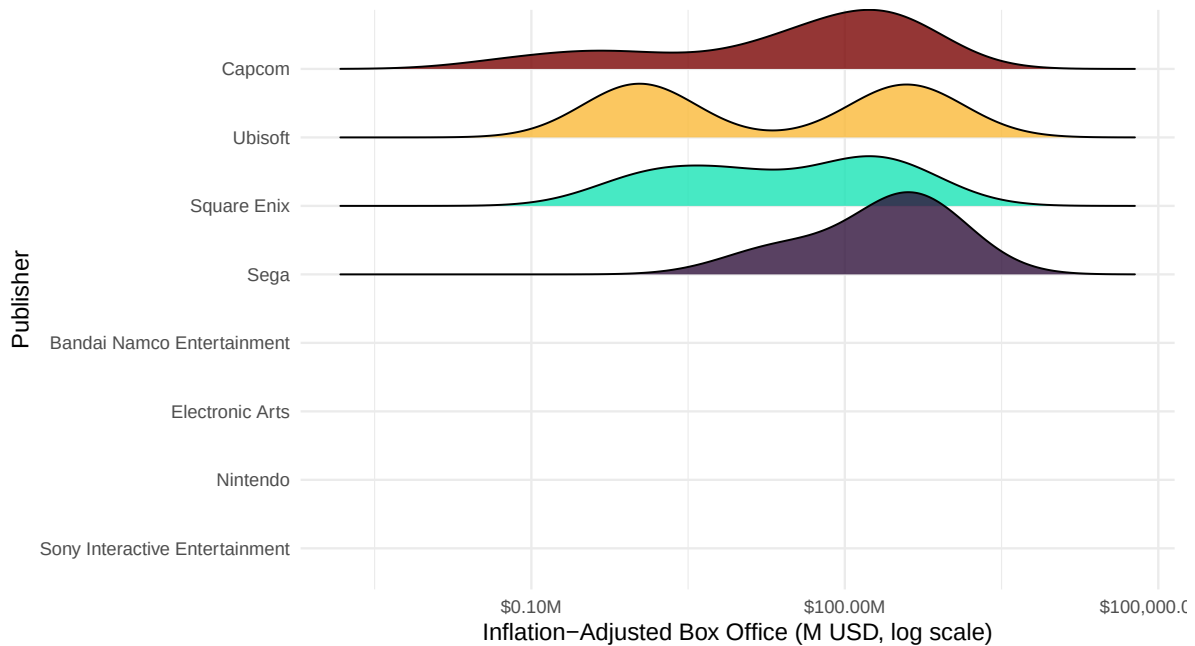


Figure 4: Distribution of inflation-adjusted box office earnings by publisher. Ridgeline density plot on a log scale.

A few things jump out. Nintendo’s distribution has a huge rightward skew from the *Super Mario* films but also includes lower earners. Capcom, Square Enix, and Sega each show multiple modes, reflecting the uneven performance of their different franchises. The earnings distributions overlap heavily across publishers of very different sizes, which backs up what we saw in Figure 3.

Key findings for Q2: A publisher’s yearly revenue does not predict box office success very well. The biggest publishers do land occasional hits (Nintendo’s *Super Mario* films), but smaller publishers like Capcom and Square Enix have put up comparable numbers. Which IP gets adapted and who makes the film seem to matter more than the size of the company behind it.

Q3: Franchise Fatigue

Our third question: do video game film franchises burn out? That is, do later installments make less money and get worse reviews? This pattern shows up a lot in regular Hollywood franchises but hasn’t been studied much for video game adaptations.

We found franchises by pulling the first meaningful word from each title (after dropping leading “The,” “A,” and “An”) and grouping films that share that word. We numbered installments by release date within each group. We only kept franchises with three or more theatrical films. This gave us *Resident Evil*, *Pokemon*, *Mortal Kombat*, *Silent Hill*, and several others.

Figure 5 plots inflation-adjusted box office by installment, with individual trend lines per franchise and an overall LM trend.

```
`geom_smooth()` using formula = 'y ~ x'
```

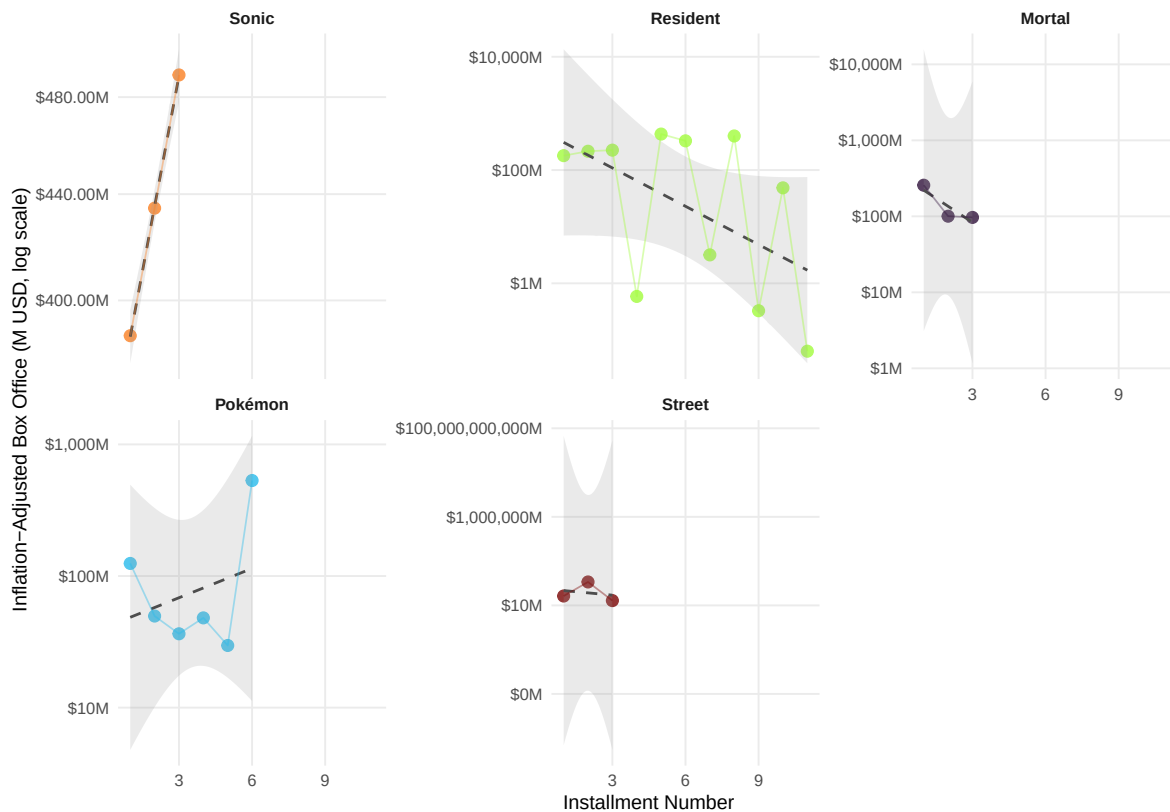


Figure 5: Inflation-adjusted box office by installment number for major video game film franchises (3+ films). Faceted by franchise with log-scale y-axis.

Figure 5 gives a mixed picture. The overall LM line slopes gently down, which fits the fatigue idea. But the individual franchises tell different stories. *Resident Evil*, with six theatrical films, shows a clear drop across the series. *Pokemon* films are more up and down: strong early entries, then a slide later.

Figure 6 checks whether Rotten Tomatoes scores follow the same pattern.

``geom_smooth()`` using formula = 'y ~ x'

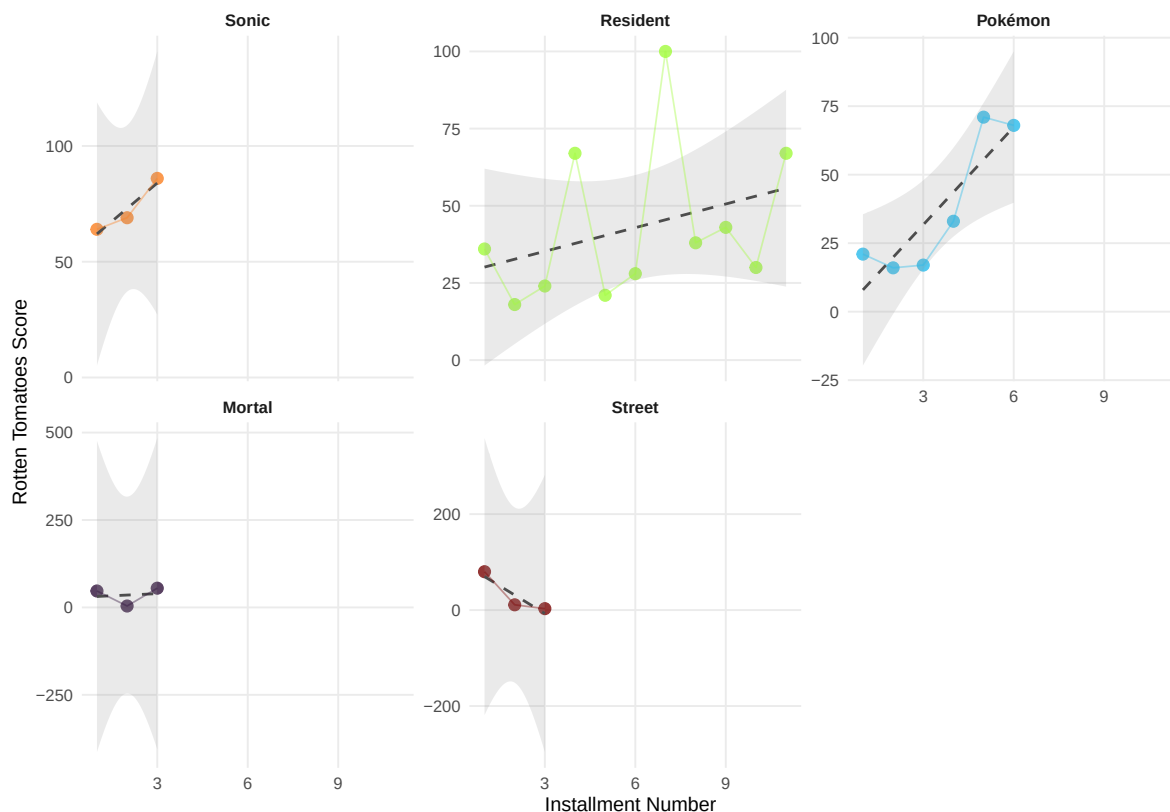


Figure 6: Rotten Tomatoes critic score by installment number for major video game film franchises.

Critic scores drop more clearly than box office. The overall LM line angles down more sharply. This means that even when later installments hold their audience or grow at the box office (maybe from built-in loyalty or nostalgia), critics get harsher with each new entry. *Resident Evil* is the clearest example: critics scored later films much lower than the 2002 original.

Figure 7 gives a different view: a tile heatmap where darker colors mean higher box office, making it easy to compare franchises across installments.

The heatmap in Figure 7 makes the fatigue pattern easy to spot. For *Resident Evil* and *Silent Hill*, the color fades from left (early installments) to right (later ones). Smaller franchises with only a few installments do not show any clear direction.

Key findings for Q3: Franchise fatigue looks real, but it hits differently depending on what you measure and which franchise you are looking at. Box office dips a bit across installments

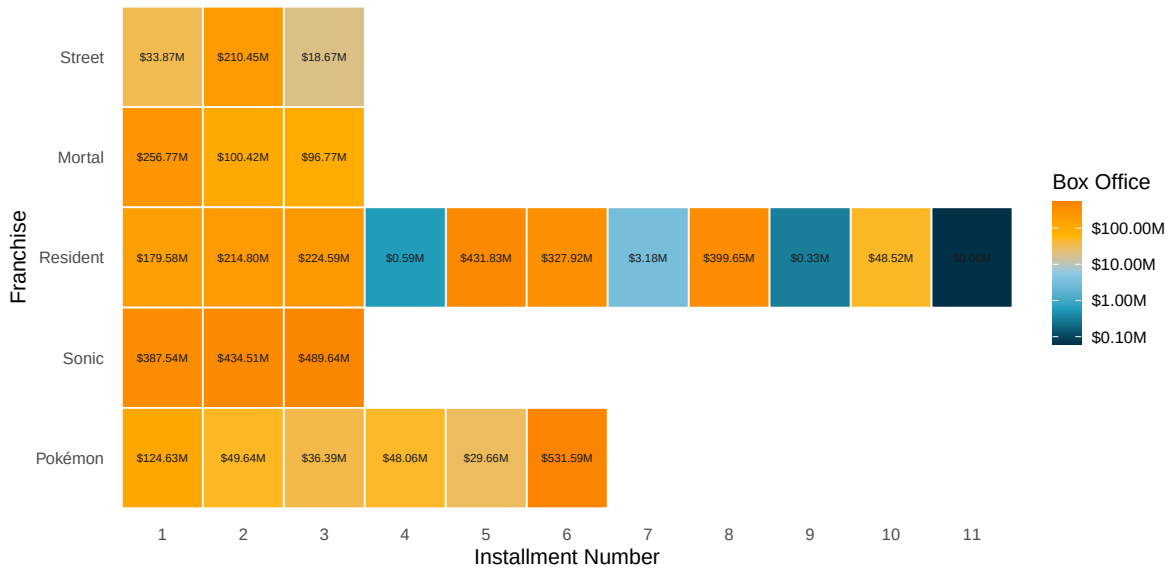


Figure 7: Franchise $\times$ installment box office heatmap. Color intensity represents inflation-adjusted box office on a log scale.

on average. Critic scores fall harder and more consistently. *Resident Evil*, the biggest franchise in the dataset with six theatrical films, shows fatigue in both money and reviews.

Correlation Analysis

Figure 8 shows a correlation matrix of all the key numeric variables. It uses a diverging red-white-teal color scale (red = negative, teal = positive) with circle size matched to the correlation strength.

```
Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
i Please use tidy evaluation idioms with `aes()`.
i See also `vignette("ggplot2-in-packages")` for more information.
i The deprecated feature was likely used in the ggcorrplot package.
  Please report the issue at <https://github.com/kassambara/ggcorrplot/issues>.
```

A few things worth pointing out from Figure 8. Budget and box office have a moderate positive correlation (r around 0.4 to 0.5). More expensive films tend to earn more, but not always. A big budget is not a guarantee. Rotten Tomatoes and Metacritic are tightly correlated (r

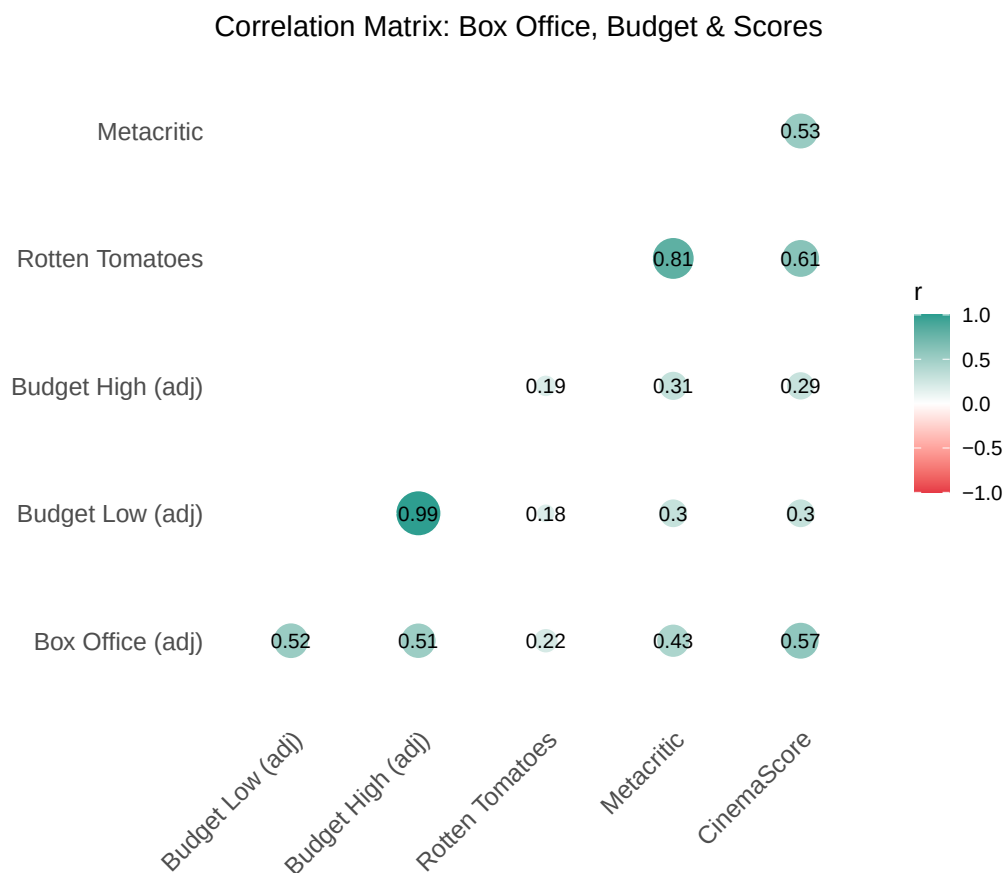


Figure 8: Correlation matrix of box office, budget, and review score variables. Diverging red to white to teal color scale with circle areas proportional to correlation magnitude.

around 0.85), which makes sense since both aggregate critic reviews. CinemaScore (audience) has a mild positive correlation with both critic metrics (r around 0.3 to 0.4), matching what we found in Q1: audiences and critics move in the same general direction but at different levels. The weakest correlation in the matrix is between budget and CinemaScore (r around 0.11). Spending more money on a film does not seem to make audiences like it more.

Findings / Conclusions

We looked at three questions about video game film adaptations across 1986 to 2024. Here is what we found:

Q1: Critic-Audience Agreement. Audiences give video game films higher scores than critics do, by about 10 to 20 points on a 0 to 100 scale. That gap has shrunk a little since 2010. All score types drifted up over the same period.

Q2: Publisher Size and Box Office. A publisher's yearly revenue does not tell you much about how well its film adaptations will do. The biggest publishers land hits sometimes (Nintendo's *Super Mario* films), but smaller publishers get comparable results. The IP itself and the people making the film seem to matter more than the company's size.

Q3: Franchise Fatigue. Video game film franchises do show signs of fatigue, especially in critic scores, which fall more steeply across installments than box office does. *Resident Evil*, with six theatrical films, shows this clearly across both dimensions.

Stepping back, these films have always been received coldly by critics and warmly by audiences. That gap might be closing as the industry figures out what works. Money seems to follow the specific property and execution more than the publisher's bank account. And franchise building has real limits: the returns shrink with each new installment.

Limitations & Future Work

A few things to keep in mind when reading these results:

- **Exchange Rates:** We used fixed approximate rates. Actual historical daily rates would be more accurate, especially for Japanese and Hong Kong films where exchange rates moved a lot over the decades.
- **CPI Data:** We used annual CPI-U averages. Monthly adjustment would tighten the inflation correction for films released early or late in a given year.
- **Missing Data:** A lot of films in the original dataset are missing box office numbers, review scores, or both. Older and smaller releases are hit hardest. Our analysis only uses films with complete data, which probably skews toward bigger, more documented releases.

- **Publisher Revenue Matching:** Only some publishers in the film dataset matched the Wikipedia list. Many smaller or defunct publishers were left out of Q2, which limits how broadly our findings apply.
- **Franchise Detection:** Our first-word method is crude. It might lump unrelated films together if they share a first word, and it misses sequels where the first word changes. A better approach would use a more careful regex or manual franchise coding.
- **Causality:** All the analyses in this project are correlational. We cannot say, for example, that publisher size causes higher box office, or that a few successful films inflate publisher revenue numbers.

Future work could go a few directions: adding genre and rating data to control for film traits, building a predictive model for box office from publisher and critic variables, looking at release strategies (staggered vs. simultaneous), or checking whether the timing between a game's release and its film adaptation affects the box office.

AI Use Statement

Harsh Rathi: Used AI to ask for suggestions on improving the report, to compare the draft against the example qmd and the rubric for gaps, to format code into clean aligned blocks, and to create the rubric and example qmd documents in the dev folder. Also used AI to debug the GitHub CI workflow. No code completion or generation was used.

Charles Bupp: [Describe how you used AI tools in your contributions]

Mason Mitchell: [Describe how you used AI tools in your contributions]

References

- TidyTuesday (2026). Game Films dataset. <https://github.com/rfordatascience/tidytuesday/tree/main/06-09>
- U.S. Bureau of Labor Statistics. Consumer Price Index for All Urban Consumers (CPI-U), series CUUR0000SA0, annual averages 1986 to 2024. Retrieved from <https://www.bls.gov/cpi/>
- Wikipedia contributors. "List of largest video game companies by revenue." Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/List_of_largest_video_game_companies_by_revenue
- R Core Team (2024). R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria.
- Wickham, H., et al. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686.

Code Appendix

Chunk setup:

```
library(tidyverse)
library(readr)
library(janitor)
library(scales)
library(rvest)
library(lubridate)
library(ggcorrplot)
library(patchwork)
library(ggthemes)
library(kableExtra)
library(viridis)

ggplot2::theme_set(theme_minimal(base_size = 12))
```

Chunk provenance-load:

```
game_films <- readr::read_csv(
  "https://raw.githubusercontent.com/rfordatascience/tidytuesday/main/data/2026/2026-06-09/g
  show_col_types = FALSE
) |>
  janitor::clean_names()
```

Chunk currency-table:

```
fx_rates <- tibble::tribble(
  ~currency, ~to_usd,
  "$",      1,
  "¥",      0.0091,
  "£",      1.30,
  "HK$",    0.128,
  "NT$",    0.033
)

fx_rates |>
  rename(Currency = currency, `USD per Unit` = to_usd) |>
  kable(caption = "Exchange rates used for currency standardization", digits = 4) |>
  kable_styling(bootstrap_options = c("striped", "condensed"), full_width = FALSE)
```


Chunk currency-convert:

```
game_films <- game_films |>
  left_join(fx_rates, by = c("worldwide_box_office_currency" = "currency")) |>
  mutate(worldwide_box_office_usd = worldwide_box_office * to_usd) |>
  select(-to_usd) |>
  left_join(fx_rates, by = c("budget_currency" = "currency")) |>
  mutate(
    budget_low_usd = budget_low * to_usd,
    budget_high_usd = budget_high * to_usd
  ) |>
  select(-to_usd)
```

Chunk cpi-data:

```
cpi <- tibble::tribble(
  ~year, ~cpi,
  1986, 109.6, 1987, 113.6, 1988, 118.3, 1989, 124.0,
  1990, 130.7, 1991, 136.2, 1992, 140.3, 1993, 144.5,
  1994, 148.2, 1995, 152.4, 1996, 156.9, 1997, 160.5,
  1998, 163.0, 1999, 166.6, 2000, 172.2, 2001, 177.1,
  2002, 179.9, 2003, 184.0, 2004, 188.9, 2005, 195.3,
  2006, 201.6, 2007, 207.3, 2008, 215.3, 2009, 214.5,
  2010, 218.1, 2011, 224.9, 2012, 229.6, 2013, 233.0,
  2014, 236.7, 2015, 237.0, 2016, 240.0, 2017, 245.1,
  2018, 251.1, 2019, 255.7, 2020, 258.8, 2021, 271.0,
  2022, 292.7, 2023, 304.7, 2024, 313.7
)

cpi_2024 <- cpi$cpi[cpi$year == 2024]

game_films <- game_films |>
  mutate(release_year = lubridate::year(release_date)) |>
  left_join(cpi, by = c("release_year" = "year")) |>
  mutate(
    inflation_factor = cpi_2024 / cpi,
    worldwide_box_office_usd_adj = worldwide_box_office_usd * inflation_factor,
    budget_low_usd_adj = budget_low_usd * inflation_factor,
    budget_high_usd_adj = budget_high_usd * inflation_factor
  ) |>
  select(-cpi, -inflation_factor)
```

Chunk theatrical-filter:

```
game_films <- game_films %>%
  filter(category == "Theatrical releases")
```

Chunk cinema-convert:

```
cinema_score_conversion <- tibble::tribble(
  ~cinemascore_letter, ~to_num,
  "N/A",             NA,
  "A+",             100,  "A",  91.6666,  "A-",  83.3333,
  "B+",             74.9999, "B",  66.6666,  "B-",  58.3333,
  "C+",             49.9999, "C",  41.6666,  "C-",  33.3333,
  "D+",             24.9999, "D",  16.6666,  "D-",   8.3333,
  "F",              0
)

game_films <- game_films %>%
  left_join(cinema_score_conversion, join_by("cinema_score" == "cinemascore_letter"))

game_films <- game_films %>%
  mutate(cinema_score = to_num) %>%
  select(-starts_with("to_num"))
```

Chunk final-summary:

```
game_films |>
  summarise(
    `Total Films` = n(),
    `Films with Box Office Data` = sum(!is.na(worldwide_box_office_usd_adj)),
    `Films with RT Score` = sum(!is.na(rotten_tomatoes)),
    `Films with Metacritic` = sum(!is.na(metacritic)),
    `Films with CinemaScore` = sum(!is.na(cinema_score)),
    `Date Range` = paste(range(release_date, na.rm = TRUE), collapse = " to ")
  ) |>
  kable(
    caption = "Overview of the cleaned dataset after all preparation steps",
    digits = 0,
    align = "l"
  ) |>
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"), full_width = FALSE, ,
```

Chunk tbl-summary-stats:

```

game_films |>
  select(
    worldwide_box_office_usd_adj,
    budget_high_usd_adj,
    rotten_tomatoes,
    metacritic,
    cinema_score
  ) |>
  summarise(across(everything(), list(
    Mean   = ~mean(.x, na.rm = TRUE),
    Median = ~median(.x, na.rm = TRUE),
    SD     = ~sd(.x, na.rm = TRUE),
    Min    = ~min(.x, na.rm = TRUE),
    Max    = ~max(.x, na.rm = TRUE)
  ))) |>
  pivot_longer(everything(), names_to = c("Variable", "Statistic"), names_pattern = "(.*)_(.*)",
  pivot_wider(names_from = Statistic, values_from = value) |>
  mutate(Variable = recode(Variable,
    worldwide_box_office_usd_adj = "Box Office (adj. USD)",
    budget_high_usd_adj          = "Budget High (adj. USD)",
    rotten_tomatoes              = "Rotten Tomatoes",
    metacritic                   = "Metacritic",
    cinema_score                 = "CinemaScore"
  )) |>
  kable(digits = 1, align = c("l", rep("r", 5))) |>
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"), full_width = FALSE)
  row_spec(0, bold = TRUE)

```

Chunk fig-q1-scatter:

```

aud_v_cri <- game_films %>%
  filter(!is.na(rotten_tomatoes) | !is.na(metacritic), !is.na(cinema_score)) %>%
  pivot_longer(cols = rotten_tomatoes:cinema_score, names_to = "critic_type", values_to = "score")
  select(title, critic_type, score, release_date)

critic_colors <- c(
  "rotten_tomatoes" = "#E63946",
  "metacritic"      = "#457B9D",
  "cinema_score"    = "#2A9D8F"
)

critic_labels <- c(

```

```

"rotten_tomatoes" = "Rotten Tomatoes (Critic)",
"metacritic"      = "Metacritic (Critic)",
"cinema_score"    = "CinemaScore (Audience)"
)

aud_v_cri %>%
  ggplot(aes(x = release_date, y = score, color = critic_type)) +
  geom_point(alpha = 0.5, size = 2.5) +
  geom_smooth(se = TRUE, method = "loess", span = 0.6, alpha = 0.15) +
  scale_color_manual(values = critic_colors, labels = critic_labels) +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y") +
  labs(x = "Release Date", y = "Score", color = "Score Type") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom", panel.grid.minor = element_blank())

```

Chunk fig-q1-gap:

```

aud_v_cri_gap <- game_films %>%
  filter(!is.na(rotten_tomatoes) | !is.na(metacritic), !is.na(cinema_score)) %>%
  mutate(
    avg_critic = rowMeans(cbind(rotten_tomatoes, metacritic), na.rm = TRUE),
    gap = avg_critic - cinema_score
  ) %>%
  filter(!is.na(gap))

aud_v_cri_gap %>%
  ggplot(aes(x = release_date, y = gap)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40", linewidth = 0.8) +
  geom_point(aes(fill = gap > 0), size = 2.5, alpha = 0.7, shape = 21, color = "white") +
  geom_smooth(method = "loess", se = TRUE, color = "#E63946", fill = "#E63946", alpha = 0.15) +
  scale_fill_manual(
    values = c("TRUE" = "#2A9D8F", "FALSE" = "#E76F51"),
    labels = c("TRUE" = "Critics Higher", "FALSE" = "Audience Higher")
  ) +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y") +
  labs(x = "Release Date", y = "Gap (Critic-Audience)", fill = "Direction") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom", panel.grid.minor = element_blank())

```

Chunk tbl-publisher-performance:

```

# Scrape and join publisher data
company_url <- "https://en.wikipedia.org/wiki/List_of_largest_video_game_companies_by_revenue"
TablesRedun <- company_url |>
  read_html() |>
  html_elements("table") |>
  html_table()
top_50 <- TablesRedun[[2]]

top_50_clean <- clean_names(top_50) %>%
  mutate(revenue_usd_billions = parse_number(revenue_usd)) %>%
  select(!c(ref, revenue_usd))

pub_joined <- as.data.frame(top_50_clean) %>%
  right_join(game_films, join_by("company" == "original_game_publisher"), copy)

table_data <- pub_joined %>%
  filter(!is.na(worldwide_box_office_usd_adj)) %>%
  group_by(company) %>%
  summarise(
    `# Films` = n(),
    `Median Box Office (M USD)` = round(median(worldwide_box_office_usd_adj) / 1e6, 1),
    `Mean Box Office (M USD)` = round(mean(worldwide_box_office_usd_adj) / 1e6, 1),
    `Mean RT Score` = round(mean(rotten_tomatoes, na.rm = TRUE), 1),
    `Mean CinemaScore` = round(mean(cinema_score, na.rm = TRUE), 1),
    .groups = "drop"
  ) %>%
  filter(`# Films` >= 3) %>%
  arrange(desc(`Mean Box Office (M USD)`))

table_data %>%
  kable(digits = 1, align = c("l", rep("r", 5))) %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"), full_width = FALSE) %>%
  row_spec(0, bold = TRUE, color = "white", background = "#2A9D8F")

```

Chunk fig-q2-multivariate:

```

pub_joined %>%
  filter(!is.na(worldwide_box_office_usd_adj), !is.na(revenue_usd_billions), !is.na(budget_high_usd_adj))
ggplot(aes(x = revenue_usd_billions,
            y = worldwide_box_office_usd_adj / 1e6,
            color = company,
            size = budget_high_usd_adj / 1e6)) +

```

```

geom_point(alpha = 0.75) +
geom_smooth(aes(group = 1), method = "lm", se = TRUE, color = "gray30",
             linewidth = 0.8, linetype = "dashed", alpha = 0.2) +
scale_color_viridis_d(option = "turbo", guide = guide_legend(ncol = 2)) +
scale_size_continuous(name = "Budget (M USD)", labels = dollar_format(), range = c(2, 10))
scale_x_log10(labels = dollar_format(suffix = "B")) +
scale_y_log10(labels = dollar_format(suffix = "M")) +
labs(
  x = "Yearly Company Revenue (USD Billions, log scale)",
  y = "Inflation-Adjusted Box Office (M USD, log scale)",
  color = "Publisher"
) +
theme_minimal(base_size = 13) +
theme(legend.position = "bottom", panel.grid.minor = element_blank())

```

Chunk fig-q2-ridge:

```

pub_joined %>%
  filter(!is.na(worldwide_box_office_usd_adj), !is.na(revenue_usd_billions)) %>%
  mutate(company = fct_reorder(company, revenue_usd_billions, .desc = TRUE)) %>%
  ggplot(aes(x = worldwide_box_office_usd_adj / 1e6, y = company, fill = company)) +
  geom_density_ridges(alpha = 0.8, scale = 1.2) +
  scale_fill_viridis_d(option = "turbo", guide = "none") +
  scale_x_log10(labels = dollar_format(suffix = "M")) +
  labs(x = "Inflation-Adjusted Box Office (M USD, log scale)", y = "Publisher") +
  theme_minimal(base_size = 13)

```

Chunk fig-q3-box-office:

```

MovieCount <- game_films %>%
  mutate(
    title_clean = str_remove(title, "^(The|A|An) "),
    start_word = str_extract(title_clean, "^[^ ]+")
  ) %>%
  select(title, start_word, release_date, worldwide_box_office_usd_adj, rotten_tomatoes) %>%
  filter(if_all(everything(), \(x) !is.na(x)))

MovieCountSummary <- MovieCount %>%
  group_by(start_word) %>%
  count() %>%
  filter(n >= 3) %>%

```

```

    arrange(desc(n))

MovieCountFinal <- semi_join(MovieCount, MovieCountSummary, join_by(start_word)) %>%
  group_by(start_word) %>%
  mutate(installment = rank(release_date)) %>%
  ungroup()

MovieCountFinal %>%
  ggplot(aes(x = installment, y = worldwide_box_office_usd_adj / 1e6)) +
  geom_point(aes(color = start_word), size = 3, alpha = 0.8) +
  geom_smooth(aes(group = 1), method = "lm", se = TRUE, color = "gray30",
              linetype = "dashed", linewidth = 0.8, alpha = 0.2) +
  geom_line(aes(group = start_word, color = start_word), linewidth = 0.5, alpha = 0.4) +
  scale_color_viridis_d(option = "turbo", guide = "none") +
  scale_y_log10(labels = dollar_format(suffix = "M")) +
  facet_wrap(~ reorder(start_word, -worldwide_box_office_usd_adj), scales = "free_y") +
  labs(x = "Installment Number", y = "Inflation-Adjusted Box Office (M USD, log scale)") +
  theme_minimal(base_size = 12) +
  theme(strip.text = element_text(face = "bold"), panel.grid.minor = element_blank())

```

Chunk fig-q3-rt:

```

MovieCountFinal %>%
  ggplot(aes(x = installment, y = rotten_tomatoes)) +
  geom_point(aes(color = start_word), size = 3, alpha = 0.8) +
  geom_smooth(aes(group = 1), method = "lm", se = TRUE, color = "gray30",
              linetype = "dashed", linewidth = 0.8, alpha = 0.2) +
  geom_line(aes(group = start_word, color = start_word), linewidth = 0.5, alpha = 0.4) +
  scale_color_viridis_d(option = "turbo", guide = "none") +
  facet_wrap(~ reorder(start_word, -rotten_tomatoes), scales = "free_y") +
  labs(x = "Installment Number", y = "Rotten Tomatoes Score") +
  theme_minimal(base_size = 12) +
  theme(strip.text = element_text(face = "bold"), panel.grid.minor = element_blank())

```

Chunk fig-q3-heatmap:

```

MovieCountFinal %>%
  mutate(start_word = fct_reorder(start_word, worldwide_box_office_usd_adj, .fun = max, .desc = TRUE))
  ggplot(aes(x = factor(installment), y = start_word, fill = worldwide_box_office_usd_adj / 1e6)) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(aes(label = scales::dollar(worldwide_box_office_usd_adj / 1e6, suffix = "M")),

```

```

      size = 2.5, color = "gray10") +
scale_fill_gradientn(
  colors = c("#023047", "#219EBC", "#8ECAE6", "#FFB703", "#FB8500"),
  labels = dollar_format(suffix = "M"),
  na.value = "gray90",
  trans = "log10"
) +
labs(x = "Installment Number", y = "Franchise", fill = "Box Office") +
theme_minimal(base_size = 13) +
theme(panel.grid = element_blank(), axis.ticks = element_blank())

```

Chunk fig-correlation:

```

cor_vars <- game_films %>%
  select(
    worldwide_box_office_usd_adj,
    budget_low_usd_adj,
    budget_high_usd_adj,
    rotten_tomatoes,
    metacritic,
    cinema_score
  ) %>%
  rename(
    `Box Office (adj)` = worldwide_box_office_usd_adj,
    `Budget Low (adj)` = budget_low_usd_adj,
    `Budget High (adj)` = budget_high_usd_adj,
    `Rotten Tomatoes` = rotten_tomatoes,
    `Metacritic` = metacritic,
    `CinemaScore` = cinema_score
  )

cor_matrix <- cor(cor_vars, use = "pairwise.complete.obs")

ggcorrplot(
  cor_matrix,
  method = "circle",
  type = "lower",
  lab = TRUE,
  lab_size = 3.5,
  colors = c("#E63946", "white", "#2A9D8F"),
  title = "Correlation Matrix: Box Office, Budget & Scores",
  ggtheme = theme_minimal(base_size = 12),

```



```
legend.title = "r",  
outline.color = "white"  
) +  
theme(panel.grid.major = element_blank())
```